
Weave of Formal Thought

Alexandre Bouayad
alex.bouayad@gmail.com

Abstract

Large language models (LLMs) attain remarkable surface fluency on code, yet they do not formally guarantee the syntactic validity of their output, nor do they typically leverage the hierarchical structure that defines the target language. While existing constrained-decoding frameworks (e.g., XGrammar, Guidance, Outlines, SynCode) offer a solution to the former, they predominantly operate under rigid assumptions that preclude critical lexical mechanisms relied upon by modern parsers — including context-sensitive lexing (e.g., Pythonic indentation), maximal-munch tokenization, and keyword extraction — and only approximate the masking of invalid subword tokens, sacrificing completeness. For the latter, modern code LLMs inject grammatical structure during training through predetermined policies rather than learning which structural information to expose. In this work, we introduce *Weave of Formal Thought* (WoFT), an overarching paradigm that unites rigorous syntactic validation with learned structural representations. First, we present a formal engine and constrained decoder that is sound and complete with respect to the full Tree-sitter specification by augmenting generalized LR (GLR) parsing with a novel *speculative-lexing* construction that maintains concurrent lexer-state hypotheses synchronized with the GLR graph-structured stack; the decoder admits every subword token that extends to a valid program prefix and rejects every token that does not. Second, we present a latent-variable fine-tuning method that trains the language model to interleave non-terminal grammar symbols directly into the generation process. Utilizing the reweighted wake-sleep (RWS) algorithm to optimize the importance-weighted evidence lower bound (IW-ELBO) of the surface text, the model learns to selectively retain formal derivations as an adaptive structural scratchpad. Across ten diverse programming languages spanning multiple paradigms (Python, C, C#, Go, Java, Rust, TypeScript, Haskell, Bash, HTML), fine-tuning StarCoder2-3B with our RWS objective consistently outperforms text-only SFT baselines, achieving up to a 14.9% relative reduction in surface-token cross-entropy (11.6% average at convergence, 14.6% at iso-step), demonstrating that discretionary latent syntax recovers critical structural information that flat autoregressive training discards. Our code and implementation are publicly available at <https://github.com/alexbouayad/formal>.

1 Introduction

Modern autoregressive large language models (LLMs) trained on code attain remarkable surface fluency: they reproduce idiomatic syntax, identifier conventions, and library-specific patterns with such fidelity that the generated text is, on average, statistically indistinguishable from human-written programs. Yet this competence is brittle in two distinct senses. First, fluency is not formal correctness: even a state-of-the-art code model produces output that fails to parse, type-check, or compile at non-trivial rates, necessitating downstream filtering or repair pipelines whose cost grows with the strictness of the target specification. Second, and more fundamentally, the model has no *explicit* access to the hierarchical structure that *defines* the language. Every program admits a unique, finite, and mathematically rigorous abstract syntax tree (AST) derivation; this derivation is precisely the

information that would let the model plan a function before writing its body, allocate a loop variable before its first use, or close a scope before opening a new one. Standard autoregressive training, by collapsing the joint distribution of text and structure onto the marginal over text alone, forces the model to *re-discover* this hierarchy implicitly at every forward pass.

Recent attempts to inject explicit reasoning into language models have largely proceeded in the opposite direction. Chain-of-thought (CoT) prompting [1] and its internal variants—Pause Tokens [2], Quiet-STaR [3], and continuous-latent reasoning [4]—grant the model additional intermediate compute, but the reasoning trajectory itself is either free-form natural language or unconstrained latent vectors. Neither offers a verifier that can reject ill-formed reasoning steps, and neither aligns with the discrete, hierarchical structure that programming languages already provide. The semantic planning gap is thus a neuro-symbolic gap: the structure of the target language exists, formally and unambiguously, but is exploited neither at training time nor at decoding time.

A parallel line of work—constrained decoding—attacks the correctness side of the problem by masking the vocabulary at each step to the tokens consistent with a partial parse [5, 6, 7, 8, 9, 10, 11]. These engines have driven structured generation latency close to that of unconstrained decoding, but their formal coverage has lagged behind the languages practitioners actually wish to generate.

To bridge this neuro-symbolic gap, we introduce *Weave of Formal Thought* (WoFT), a unified paradigm that integrates rigorous syntactic validation with learned structural representations. WoFT operates through two complementary components: a formal inference engine (metaphorically, the “loom”) and a latent-variable fine-tuning method (the “weaver”).

First, we present a language-model-agnostic formal engine and constrained decoder that is sound and complete with respect to the full Tree-sitter specification [12]. By augmenting Generalized LR (GLR) parsing [13] with a speculative-lexing construction, our engine maintains concurrent lexer-state hypotheses synchronized with a GLR graph-structured stack. This approach natively supports context-sensitive lexing (such as Pythonic indentation), maximal-munch tokenization, and declarative ambiguity resolution. The resulting decoder admits every subword token that extends to a valid program prefix and rejects every token that does not.

Second, we introduce a latent-variable fine-tuning method that trains the language model to interleave non-terminal grammar symbols directly into the generation process. Rather than forcing the model to emit a fixed, deterministic syntax trace, we treat formal derivations as discrete latent variables. We train the model using the reweighted wake-sleep (RWS) algorithm [14] to optimize the importance-weighted evidence lower bound (IW-ELBO) [15] of the surface text. This enables the model to utilize formal non-terminals as an adaptive structural scratchpad, retaining them only when they effectively compress future surface tokens. Empirically, fine-tuning StarCoder2-3B [16] across ten diverse programming languages (Python, C, C#, Go, Java, Rust, TypeScript, Haskell, Bash, HTML) with our RWS objective consistently outperforms text-only SFT baselines, reducing surface-token cross-entropy by an average of 11.6% at final convergence (up to 14.9% on C) and 14.6% at iso-step. This demonstrates that discretionary latent syntax recovers critical structural information that flat autoregressive training discards. Our complete implementation, including the formal engine and RWS training pipeline, is publicly available at <https://github.com/alexbouayad/formal>.

The remainder of the paper is organized as follows. Section 2 reviews related work across constrained decoding, latent structure in language modeling, and internal reasoning. Section 3 details the design and theoretical guarantees of our formal engine. Section 4 presents our latent-variable formulation, the RWS optimization objective, and the training architecture. Section 5 details our experimental setup and reports empirical results on surface token modeling. Finally, Section 6 concludes with our research vision and next steps.

2 Related Works

2.1 Constrained decoding and formal grammars

Constrained decoding intervenes during autoregressive generation to guarantee that language model outputs satisfy a formal-language specification, typically by masking the model’s vocabulary at each step to admit only tokens consistent with a partial parse. PICARD [5] introduced incremental parser-guided decoding for SQL; Outlines [6] and SynCode [7] generalized this approach to arbitrary regular and context-free languages by compiling the grammar into finite-state machines and pushdown

automata, respectively, and by precomputing per-token masks offline. Recent engines push these ideas toward production-scale latency. XGrammar [8, 9] partitions the vocabulary into context-independent and context-dependent subsets to amortize mask construction across decoding steps, layered on top of byte-level Earley [17] and pushdown-automaton recognizers. Guidance [11] and its Rust backend LLGuidance [10] fuse derivative-based parsing directly with the decoding loop.

A complementary line of work identifies the misalignment between subword vocabularies and formal language lexemes as a primary source of correctness failures. DOMINO [18] explicitly addresses this gap by combining offline precomputation with speculative decoding to enforce constraints in a subword-aligned fashion, achieving near-zero overhead on regex- and CFG-bounded targets. We share DOMINO’s diagnosis but operate one level higher in the compiler stack: rather than speculating over subword sequences within a fused regex/CFG state machine, we maintain multiple concurrent lexer states alongside a Generalized LR [13] graph-structured stack. This preserves a separate, mature lexer at the granularity of formal lexemes and supports stateful lexical behaviors that scannerless engines cannot express.

In contrast, our formal engine (Section 3) directly integrates the full Tree-sitter framework [12]. We resolve the subword-alignment gap by maintaining speculative-lexing hypotheses synchronized with a GLR graph-structured stack, achieving soundness and completeness over production grammars.

2.2 Latent structure in language modeling

The integration of explicit syntactic structure into language modeling has a rich history in natural language processing. Early structured language models [19, 20, 21] demonstrated that conditioning next-token prediction on partial parse trees improved perplexity. With the advent of deep learning, frameworks like Recurrent Neural Network Grammars (RNNGs) [22] and latent Probabilistic Context-Free Grammars (PCFGs) [23] attempted to treat these syntactic trees as latent variables. These models optimized the marginal likelihood of the observed text by marginalizing over all possible tree structures using dynamic programming (e.g., the Inside-Outside algorithm).

While theoretically elegant, these historical latent-structure models faced computational scaling limits. Exact marginalization over parse trees scales cubically with sequence length, making it computationally intractable for the large vocabularies and context windows of modern Transformers. Consequently, the standard language modeling paradigm abandoned explicit latent syntactic structures, relying entirely on self-attention mechanisms to implicitly learn flat representations of code and text.

More recent work has pushed internal reasoning further into latent space. Coconut [4] replaces discrete chain-of-thought tokens with continuous hidden-state vectors fed back as input embeddings, allowing the model to encode multiple alternative reasoning steps simultaneously and perform breadth-first search over reasoning paths. While this circumvents the textual-coherence overhead of natural-language reasoning, the resulting latents are continuous vectors with no symbolic semantics; they cannot be validated, interpreted, or composed by an external verifier. Our framework occupies a complementary point in the design space: latents are discrete non-terminal symbols drawn from a formal grammar, and every latent trajectory is verified against a Tree-sitter GLR oracle. This trades Coconut’s continuous geometry for symbolic groundedness, hierarchical interpretability, and the ability to mathematically constrain the reasoning trajectory to valid AST derivations. By modeling formal AST derivations as discrete latents trained via reweighted wake-sleep (RWS), our framework (Section 4) enables autoregressive Transformers to explicitly leverage hierarchical syntax without the computational bottleneck of earlier structured models.

2.3 Internal chain-of-thought and reasoning

Standard chain-of-thought (CoT) prompting [1] improves the semantic planning of language models by allocating additional intermediate compute steps (forward passes) before yielding a final answer. Recently, there has been a push toward internal or implicit reasoning—allowing models to “think” in a hidden scratchpad that is omitted from the final output. Techniques like Pause Tokens [2] insert dummy tokens to delay generation, while Quiet-STaR [3] trains models via REINFORCE to generate hidden natural language rationales, rewarding internal thoughts that minimize the cross-entropy of future target tokens.

Current internal reasoning frameworks rely heavily on free-form, unconstrained natural language. Because the hidden rationales are generated autoregressively without formal boundaries, they remain susceptible to semantic drift, hallucination, and an unguided search space. When writing code, unconstrained internal thoughts often fail to align with the strict topological requirements of the target algorithm.

Our framework redefines internal reasoning by replacing unconstrained natural language with strict formal grammars. By selectively retaining non-terminal symbols (e.g., `<statement>`, `</statement>`) via our proposal head, the model engages in latent syntactic planning. These non-terminals act as a rigorously structured internal scratchpad derived from the true abstract syntax tree of the code, grounding the internal reasoning process in valid syntactic blueprints.

3 The Formal Engine

The first component of the Weave of Formal Thought (WoFT) paradigm is the formal engine, which serves as the rigorous structural mechanism—metaphorically, the “loom”—that guarantees syntactic validity during generation. To successfully weave formal grammar tokens and surface text without tangling or breaking the underlying syntactic constraints, the inference engine must precisely track the grammatical state of the program. In this section, we describe the design of our formal engine, a language-model-agnostic constrained decoder that achieves soundness and completeness with respect to the full Tree-sitter specification.

3.1 GLR parsing and Tree-sitter

Deterministic LR parsers [24] are limited to unambiguous context-free grammars. When parsing non-deterministic grammars with shift/reduce or reduce/reduce conflicts, Generalized LR (GLR) parsing [13] bifurcates the parse stack to explore conflicting grammatical hypotheses in parallel, merging stacks when they reconverge or terminating them when they prove invalid.

Modern programming languages such as Python, JavaScript, and C++ require parsing mechanisms beyond pure context-free formalisms. These include lexical context sensitivity (e.g., disambiguating keywords from identifiers based on parse state), maximal-munch tokenization, and complex external lexing rules like indentation tracking. Modern parsers and parser generators integrate these advanced lexical mechanisms alongside GLR parsing to handle production source code.

Interactive software environments further require real-time, incremental parsing to maintain syntax trees during live editing [25]. Tree-sitter [12] provides an incremental GLR parsing infrastructure that rapidly updates syntax trees as code is modified.

However, while Tree-sitter supports incremental parsing for complete lexical edits in a live editor, language models generate code autoregressively via arbitrary subword tokens that split lexical boundaries. Because Tree-sitter’s underlying lexer and external scanners rely on complete token streams to enforce maximal-munch tokenization, they cannot evaluate whether an incomplete subword token represents a valid prefix of a future legal statement. To serve as an online constrained decoder, our formal engine extends Tree-sitter’s GLR framework with a speculative, subword-level execution model.

3.2 Architecture

The formal engine is structured as seven co-operating components designed to reconcile subword-level language model generation with compiler-grade lexical and syntactic parsing: a BUFFER, a dual-DFA LEXER, an external SCANNER, a graph-structured stack (STACK), a parse LATTICE, an extended GLR PARSER, and the engine FRONTEND. The architecture is language-model- and grammar-agnostic: the FRONTEND decodes emitted subwords into characters and feeds them to the BUFFER, while the remaining components operate directly over compiled Tree-sitter tables and external scanners without modification.

The primary algorithmic hurdle in constrained decoding for production grammars is the misalignment between subword boundaries and formal lexemes. Traditional compiler lexers operate under a maximal-munch assumption on complete input streams, whereas autoregressive decoding feeds

arbitrary character fragments. Our engine resolves this through a *speculative-lexing* mechanism mediated by the LATTICE and coroutine-based *scanlets*:

- **Stream buffering and transactional rollbacks:** The BUFFER maintains an append-only character stream with transactional checkpointing (`validate` and `revert`), allowing speculative tokens to be evaluated and rejected without corrupting parse state.
- **Dual-track lexing and external scanlets:** The LEXER advances two parallel DFA states (plus for contextually admissible terminals and keywords; minus for keyword-shadowed word terminals) to reproduce Tree-sitter’s keyword extraction. For context-sensitive lexing (e.g., Python indentation), the SCANNER wraps compiled C external scanners inside suspendable *scanlets* that yield execution back to the parser when hitting the buffer boundary.
- **Lattice of lexico-parse hypotheses:** Rather than maintaining only alternative parse stacks via a Graph-Structured Stack (GSS) [13], the LATTICE arranges joint lexico-parse hypotheses in a dependency graph. When the buffer is exhausted mid-lexeme, continuation vertices are superposed in the lattice, preserving multiple concurrent lexical futures and enforcing maximal munch across subword boundaries.
- **GLR dispatch and rejection sampling:** The PARSER drives GLR shifts and reductions across active lattice hypotheses via a deduplicating worklist. At each decoding step, the FRONTEND snapshots the lattice, speculatively parses candidate subwords, and admits a token if and only if at least one hypothesis survives.

Under standard well-formedness assumptions on the grammar, this construction is sound and complete with respect to the Tree-sitter specification. We provide the exhaustive, component-by-component architectural specification and an analysis of the speculative-lexing mechanics in Appendix A.

4 Weave of Formal Thought

While the formal engine provides the rigorous structural loom to ensure that language models output valid surface forms at inference time, constrained decoding alone fundamentally relies on an autoregressive policy that treats syntax as an external filter rather than an internalized representation. We hypothesize that modeling the latent syntactic derivation—the *formal thought* underlying the surface code—provides a richer training signal. To this end, the second component of the Weave of Formal Thought (WoFT) paradigm is a training method that fine-tunes a model to interleave non-terminal grammatical derivations as discrete latent variables during the generation process. Under this view, each formal token represents a thought of a *formal* nature, in the sense that the thought lives in a discretized space—or rather a collapsed projection of an internal one, continuing the quantum analogy—parameterizing the structural *form* of the language. We adopt the term *weave* to reflect that these formal thoughts are not chained sequentially prior to generation, but are instead interleaved directly into the surface text, acting as the structural weft that supports the surface code.

4.1 AST derivations as discrete latents

To explicitly model the hierarchical structure of the target language, we construct the abstract syntax tree (AST) of the surface code using a Tree-sitter parser and flatten it into a linear sequence that interleaves standard text subwords with formal grammar tokens. During a depth-first traversal of the AST, we emit formal tokens that serialize the syntactic derivation: these take the form of XML-like tags denoting non-terminal grammar rules (e.g., `<statement>`, `</statement>`) as well as syntactic fields (e.g., `[condition]`, `[/condition]`). This serialization is configurable: we can choose to include or exclude field tokens, and to emit only starting tag tokens (prefix placement), only ending tag tokens (postfix placement), or both, allowing us to flexibly control the density of the formal signal injected into the sequence.

The interleaved sequence thus comprises tokens drawn from two logically distinct vocabularies: the base text vocabulary and the formal grammar vocabulary. We treat the formal tokens as a sequence of discrete latent variables z , while the surface text tokens constitute the observed variables x . Our goal is to maximize the marginal likelihood of the surface code $p_{\theta}(x) = \sum_z p_{\theta}(x, z)$, effectively marginalizing over all possible subsets of formal tokens. During training, we represent the decision to keep or drop each formal token as an independent Bernoulli variable, yielding a sequence of binary

submasks. By selectively masking formal tokens—essentially removing them from the sequence—we enable the model to explore and marginalize over different latent derivations.

4.2 Proposal and generative joint modeling

To optimize the marginal likelihood with discrete latents, we employ an amortized variational inference framework consisting of two components: a proposal (or inference) distribution $q_\phi(z | x)$ and a generative model $p_\theta(x, z)$. Rather than maintaining separate networks, both distributions are parameterized by a single pre-trained base causal language model augmented with a shared parameter-efficient fine-tuning (PEFT) adapter.

We extend the vocabulary of the base model by introducing dedicated input and output embedding layers for the formal tokens. The shared model processes the mixed sequence of text and formal tokens to produce contextualized hidden states. From these shared representations, the architecture diverges into two distinct linear heads.

For the *proposal head* (ϕ), a one-dimensional linear projection maps the hidden states of the formal tokens to token-level logits. These logits parameterize independent Bernoulli distributions over the formal mask, dictating the probability of keeping each formal token in the sequence. For the *generative head* (θ), the standard language modeling output embeddings are used for surface text tokens, while the dedicated formal output embeddings are used for formal tokens.

By sharing the underlying base model and its PEFT adapter, and routing only through dedicated linear heads, we efficiently train both ϕ and θ with minimal memory and computational overhead.

4.3 Optimization via reweighted wake-sleep

While techniques such as the straight-through estimator (STE) [26] or continuous relaxations, e.g., Gumbel-Softmax [27, 28], are often used to circumvent the non-differentiability of discrete variables, they are fundamentally incompatible with our architecture. In our model, the discrete formal mask dictates more than just the weighting of tokens; it dictates the physical compaction of the sequence. When a formal token is dropped, all subsequent tokens shift left. Consequently, this single discrete choice not only alters the overall sequence length and the positional encodings, but also cascades through the causal attention mechanism to perturb the logits of every subsequent token. Because this constitutes discrete stochastic control flow—where the computational graph itself depends on the sampled latents [29]—continuous relaxations are inapplicable.

Instead, we optimize the importance weighted evidence lower bound (IW-ELBO) [15]. While the standard ELBO bounds the log marginal likelihood using a single sample, it often forces the proposal distribution to prematurely collapse around a single mode. The IWAE lower bound mitigates this by averaging over K independent particles:

$$\mathcal{L}_{\text{IWAE}} = \mathbb{E}_{z_1, \dots, z_K \sim q_\phi(z|x)} \left[\log \frac{1}{K} \sum_{k=1}^K \frac{p_\theta(x_k)}{q_\phi(z_k | x)} \right]$$

where x_k is the sequence with the submask applied. The IWAE objective is yielding a strictly tighter lower bound that encourages the proposal to explore multiple plausible trajectories. To optimize this objective, we employ the reweighted wake-sleep (RWS) algorithm [14], which provides a highly effective, low-variance gradient estimator specifically designed for models with discrete stochastic control flow [30, 31]. We outline one step of the training loop in algorithm 1. Notice that the IW-ELBO is not differentiated directly: instead, we perform gradient descent on the surrogate pseudo-loss $\mathcal{J}_{\text{RWS}}$. Additionally, during the wake- θ phase, we subsample a single particle proportional to its normalized importance weight rather than computing the exact weighted sum over all K particles. This particle-resampling approach significantly reduces the memory and computational footprint of the generative backward pass.

The algorithm relies strictly on the two *wake* phases (wake- θ and wake- ϕ), fully omitting the *sleep* phase present in the original wake-sleep algorithm. In standard settings, the sleep phase is usually dropped entirely because training the proposal on data “dreamed” by an untrained generative model introduces a detrimental data distribution bias [30]. While our situation is distinct since we are fine-tuning a pre-trained language model rather than training from scratch, we still omit the sleep phase because data sparsity is not a primary bottleneck in our setting. The model converges rapidly

Algorithm 1 Reweighted Wake-Sleep (RWS) Training Step

Require: Model’s proposal parameters ϕ and generative parameters θ

Require: Number of particles K

Require: Input sequence comprising x interleaved text and formal tokens

- 1: **for** $k = 1, \dots, K$ **do**
 - 2: Sample binary submask $z_k \sim q_\phi(z \mid x)$ $\triangleright$ *Particle Sampling*
 - 3: $x_k \leftarrow$ Apply submask z_k to x
 - 4: $w_k \leftarrow p_\theta(x_k)/q_\phi(z_k \mid x)$
 - 5: **end for**
 - 6: $\tilde{w}_1, \dots, \tilde{w}_K \leftarrow \text{Softmax}(\log w_1, \dots, \log w_K)$ $\triangleright$ *Normalized Weights*
 - 7: $\mathcal{J}_{\text{RWS}} \leftarrow -\sum_k \tilde{w}_k \log q_\phi(z_k \mid x)$ $\triangleright$ *Wake- ϕ Phase*
 - 8: Update ϕ via gradient descent on $\mathcal{J}_{\text{RWS}}$
 - 9: Subsample single particle $k' \sim \text{Categorical}(\tilde{w}_1, \dots, \tilde{w}_K)$ $\triangleright$ *Wake- θ Phase*
 - 10: $\mathcal{J}_{\text{RWS}} \leftarrow -\log p_\theta(x_{k'})$
 - 11: Update θ via gradient descent on $\mathcal{J}_{\text{RWS}}$
-

relative to the abundance of available surface code, rendering the sleep phase unnecessary for basic convergence. Nonetheless, future iterations could incorporate the sleep phase as a regularizer, driving the proposal to better approximate the generative posterior; this may lower the variance of the importance weights in the wake phase, thereby mitigating the self-normalized importance sampling (SNIS) bias inherent to the wake- ϕ update.

Compared to alternative discrete score-function estimators such as VIMCO [32], ARM [33], DIS-ARM [34], and OVIS [35], which scale gradient updates by unnormalized objective magnitudes and risk signal collapse near sharp local optima, RWS maintains a constant total gradient weight of 1. This provides a natural trust-region regularization that yields low-variance, stable updates under discrete stochastic control flow. We provide an extended theoretical comparison of these estimators in Appendix B.

5 WoFT Improves Surface Modeling

In this section, we empirically evaluate whether Weave-of-Formal-Thought fine-tuning via RWS improves the base language model’s ability to model surface tokens across diverse programming languages. We first detail the experimental setup, including the multi-language corpus, tokenization strategy, and optimization hyperparameters. We then present empirical results comparing the generative performance of our approach against standard supervised fine-tuning baselines on Python, followed by a cross-lingual evaluation across nine additional languages.

5.1 Experimental setup

We conduct our experiments using StarCoder2-3B [16] as the base architecture. To evaluate generality across programming paradigms, we train models across ten distinct languages: Python, C, C#, Go, Java, Rust, TypeScript, Haskell, Bash, and HTML. For each language, our training dataset consists of a 15,000-sequence subset sampled from The Stack v2 [16], specifically targeting source files between 1 and 100,000 bytes. To process these files into formal sequences, we rely on StarCoder2-3B’s subword tokenizer for the underlying text. We parse raw source code using Tree-sitter to construct an abstract syntax tree (AST), then traverse this AST in prefix order, interleaving surface text tokens with newly introduced formal tokens denoting non-terminal grammar rules. This extended formal vocabulary is appended directly to the base tokenizer. Finally, the sequences are processed with a maximum length of 16,000 tokens.

To efficiently accommodate the newly introduced formal grammar vocabulary and enable the model to internalize syntactic structure, we employ low-rank adaptation (LoRA) [36] with rank $r = 32$. Adapters are applied across all attention projections (q_proj, k_proj, v_proj, o_proj),

feed-forward layers (`c_fc`, `c_proj`), and crucially, the embedding and language modeling heads (`embed_tokens`, `lm_head`).

All models are optimized using AdamW [37] with a learning rate of 3×10^{-4} , weight decay of 0.01, and gradient clipping at maximum norm 1.0. Training is performed on an NVIDIA A40 GPU in `bfloat16` precision, leveraging PyTorch’s [38] Scaled Dot-Product Attention (SDPA) [39, 40] and gradient checkpointing [41]. For WoFT models trained via reweighted wake-sleep, we use $K = 4$ independent particles per sequence to construct the variance-reduced gradient estimators. Our implementation is built upon the Hugging Face `transformers` [42], `datasets` [43], and `peft` [44] libraries. The complete codebase is publicly available at <https://github.com/alexbouayad/formal>.

5.2 Generative performance across programming languages

To evaluate the performance of Weave-of-Formal-Thought (WoFT) fine-tuning, and to assess the efficacy of our training method, we compare the generative performance of our model trained via the reweighted wake-sleep (RWS) algorithm against standard supervised fine-tuning (SFT) baselines. All models are trained to optimize next-token prediction, measured via per-token cross-entropy loss.

Isolating latent structure on Python. To isolate the effect of our discrete latent variable formulation from the mere presence of syntactic data, we first evaluate two baselines on Python: a *Text SFT* baseline trained exclusively on the original surface text, and a *Text+Formal SFT* baseline trained on surface text interleaved with formal grammar tokens via standard autoregressive teacher-forcing. Following standard single-epoch training on deduplicated data, the smoothed online training loss (250-step running average) provides an unbiased estimator of generalization error.

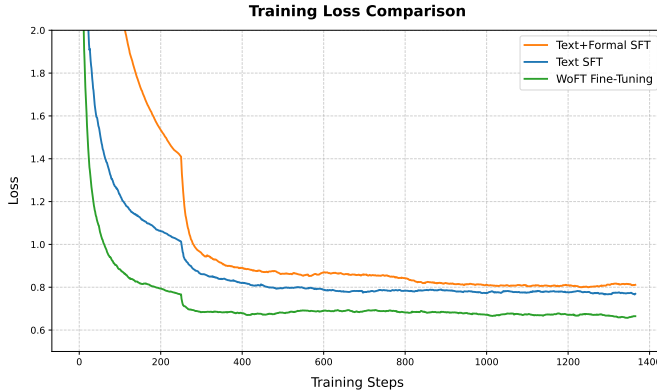


Figure 1: Smoothed online training loss (per-text-token cross-entropy) across the three fine-tuning methods on Python (running average over 250 steps). WoFT fine-tuning via RWS achieves a substantially lower surface cross-entropy (0.66) than both Text SFT (0.77) and Text+Formal SFT (0.82).

As shown in Figure 1, training WoFT with the RWS objective yields a substantial reduction in surface cross-entropy on Python, converging to 0.66 compared to 0.77 for Text SFT (a 13.6% relative reduction). Crucially, Text+Formal SFT performs worse (0.81), confirming that forcing the model to deterministically unroll trivial syntax via teacher forcing distracts capacity from high-entropy surface tokens. In contrast, WoFT selectively retains formal derivations only when they effectively compress future surface text, serving as an adaptive structural scratchpad.

Cross-lingual validation across ten languages. To verify that these structural benefits generalize beyond Python, we fine-tune StarCoder2-3B using WoFT and Text SFT across nine additional languages spanning diverse programming paradigms: C (imperative/systems), C# (object-oriented), Go (concurrent/systems), Java (object-oriented), Rust (functional/systems), TypeScript (web/multi-paradigm), Haskell (purely functional), Bash (shell scripting), and HTML (declarative/markup).

Table 1: Generative cross-entropy of StarCoder2-3B fine-tuned via Text SFT vs. Weave of Formal Thought (WoFT) across ten programming languages from The Stack v2. WoFT consistently outperforms Text SFT across every language, achieving an average relative loss reduction of 11.6% at final convergence and 14.6% at iso-step.

Language	Paradigm	Text SFT	WoFT (RWS)	Δ Final (%)	Δ Iso-step (%)
Python	Multi-paradigm / Scripting	0.7691	0.6648	13.57%	13.57%
C	Imperative / Systems	0.6902	0.5874	14.90%	15.95%
C#	Object-Oriented	0.5782	0.5205	9.98%	15.77%
Go	Concurrent / Systems	0.6021	0.5273	12.43%	15.70%
Java	Object-Oriented	0.5856	0.5204	11.14%	14.81%
Rust	Functional / Systems	0.5526	0.4929	10.80%	15.01%
TypeScript	Multi-paradigm / Web	0.6526	0.5911	9.43%	12.54%
Haskell	Purely Functional	0.8997	0.8195	8.92%	15.08%
Bash	Shell Scripting	1.1311	0.9940	12.12%	12.57%
HTML	Declarative / Markup	0.9414	0.8206	12.83%	14.97%
Average	—	0.7403	0.6538	11.61%	14.60%

Table 1 reports the results. WoFT achieves strictly lower cross-entropy across every single language without exception. At final convergence, relative reductions average 11.6%, ranging from 8.9% on Haskell to 14.9% on C. Notably, each gradient step corresponds to the exact same number of sequences trained on across both methods (10 sequences per step via gradient accumulation). What emerges is that WoFT converges on average three times faster. Evaluated at iso-step (comparing Text SFT at the exact same number of sequences trained on as WoFT’s final step), WoFT’s relative loss reduction averages 14.6% across all languages (spanning 12.5% to 16.0%). This establishes that the benefits of learned discretionary latent syntax are paradigm-invariant and universally applicable, providing superior sample efficiency alongside stronger generative modeling.

6 Conclusion

In this work, we introduced two complementary contributions to bridge the neuro-symbolic gap in code generation. First, we developed a formal inference engine that provides sound and complete constrained decoding with respect to the full Tree-sitter specification, resolving the subword-lexeme misalignment through speculative GLR parsing. Second, we proposed Weave of Formal Thought (WoFT), a latent-variable fine-tuning method that trains language models to interleave non-terminal grammar derivations as discrete latents optimized via the reweighted wake-sleep (RWS) algorithm.

Our empirical findings across ten diverse programming languages demonstrate that treating formal syntax as an adaptive structural scratchpad consistently reduces surface-token cross-entropy (averaging 11.6% at final convergence and 14.6% at iso-step) on StarCoder2-3B compared to text-only SFT baselines, while teacher-forcing full deterministic syntax traces actively degrades surface modeling.

In the near term, practical next steps focus on low-level decoder acceleration (byte-level lexing, trie-based masking, incremental parsing), refined empirical ablations, functional execution benchmarks, and online coupling of the formal engine with WoFT fine-tuning. We detail this roadmap and our broader research vision in Appendix C.

Ultimately, our results confirm that explicitly modeling syntax assists surface generation when the model is granted the discretion to prune unneeded structure. This aligns closely with established findings in cognitive science and neuroscience, where experimental evidence demonstrates that the human brain constructs hierarchical, grammatical hypotheses during real-time language processing. By treating formal syntax as both an external guarantee and an internal latent reasoning substrate, this work establishes a grounded path toward bridging structural reasoning and statistical fluency.

Acknowledgments and Disclosure of Funding

The author thanks G-Research for their support and authorization to publish this work. The author is also deeply grateful to Inria Paris for their welcoming hospitality and supportive research environment.

References

- [1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.
- [2] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. In *The Twelfth International Conference on Learning Representations*, 2024.
- [3] Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D. Goodman. Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking, 2024.
- [4] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason E Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. In *Second Conference on Language Modeling*, 2025.
- [5] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901, 2021.
- [6] Brandon T. Willard and Rémi Louf. Efficient Guided Generation for Large Language Models, 2023.
- [7] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. SynCode: LLM generation with grammar augmentation. *Transactions on Machine Learning Research*, 2025.
- [8] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ziyi Xu, Yilong Zhao, Ruihang Lai, and Tianqi Chen. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. In *Eighth Conference on Machine Learning and Systems*, 2025.
- [9] Linzhang Li, Yixin Dong, Guanjie Wang, Ziyi Xu, Alexander Jiang, and Tianqi Chen. XGrammar-2: Efficient Dynamic Structured Generation Engine for Agentic LLMs, 2026.
- [10] guidance-ai. LLGuidance: Super-fast Structured Outputs, 2025.
- [11] guidance-ai. Guidance: A guidance language for controlling large language models, 2023.
- [12] Tree-sitter contributors. Tree-sitter: An Incremental Parsing System for Programming Tools, 2026.
- [13] Masaru Tomita. An efficient augmented-context-free parsing algorithm. *Computational Linguistics*, 13:31–46, 1987.
- [14] Jörg Bornschein and Yoshua Bengio. Reweighted wake-sleep. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.
- [15] Yuri Burda, Roger Grosse, and Ruslan Salakhutdinov. Importance Weighted Autoencoders. In *The Fourth International Conference on Learning Representations*, 2016.
- [16] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Noua-mané Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. StarCoder2 and The Stack v2: The Next Generation, 2024.

- [17] Jay Earley. An efficient context-free parsing algorithm. *Communications of The Acm*, 13(2):94–102, 1970.
- [18] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Guiding LLMs the right way: Fast, non-invasive constrained generation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 3658–3673, 2024.
- [19] Ciprian Chelba and Frederick Jelinek. Exploiting syntactic structure for language modeling. In *36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics, Volume 1*, pages 225–231, 1998.
- [20] Brian Roark. Probabilistic top-down parsing and language modeling. *Computational Linguistics*, 27(2):249–276, 2001.
- [21] Eugene Charniak. Immediate-head parsing for language models. In *Proceedings of the 39th Annual Meeting of the Association for Computational Linguistics*, pages 124–131, 2001.
- [22] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 199–209, 2016.
- [23] Yoon Kim, Alexander Rush, Lei Yu, Adhiguna Kuncoro, Chris Dyer, and Gábor Melis. Un-supervised recurrent neural network grammars. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 1105–1117, 2019.
- [24] Donald E. Knuth. On the translation of languages from left to right. *Information and Control*, 8(6):607–639, 1965.
- [25] Tim Allen Wagner. *Practical Algorithms for Incremental Software Development Environments*. PhD thesis, University of California, Berkeley, 1997.
- [26] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation, 2013.
- [27] Eric Jang, Shixiang Gu, and Ben Poole. Categorical Reparameterization with Gumbel-Softmax, 2017.
- [28] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables, 2017.
- [29] John Schulman, Nicolas Heess, Theophane Weber, and Pieter Abbeel. Gradient Estimation Using Stochastic Computation Graphs, 2016.
- [30] Tuan Anh Le, Adam R. Kosiorek, N. Siddharth, Yee Whye Teh, and Frank Wood. Revisiting Reweighted Wake-Sleep for Models with Stochastic Control Flow, 2019.
- [31] Romain Lopez, Pierre Boyeau, Nir Yosef, Michael I. Jordan, and Jeffrey Regier. Decision-Making with Auto-Encoding Variational Bayes, 2020.
- [32] Andriy Mnih and Danilo J. Rezende. Variational inference for Monte Carlo objectives, 2016.
- [33] Mingzhang Yin and Mingyuan Zhou. ARM: Augment-REINFORCE-merge gradient for stochastic binary networks. In *International Conference on Learning Representations*, 2019.
- [34] Zhe Dong, Andriy Mnih, and George Tucker. DisARM: An antithetic gradient estimator for binary latent variables. In *Advances in Neural Information Processing Systems*, volume 33, pages 18637–18647, 2020.
- [35] Valentin Liévin, Andrea Dittadi, Anders Christensen, and Ole Winther. Optimal variance control of the score-function gradient estimator for importance-weighted bounds. In *Advances in Neural Information Processing Systems*, volume 33, pages 16591–16602, 2020.

- [36] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [37] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization, 2019.
- [38] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library, 2019.
- [39] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359, 2022.
- [40] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024.
- [41] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training Deep Nets with Sublinear Memory Cost. *CoRR*, abs/1604.06174, 2016.
- [42] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. HuggingFace’s Transformers: State-of-the-art Natural Language Processing, 2020.
- [43] Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, Mariama Drame, Julien Plu, Lewis Tunstall, Joe Davison, Mario Šaško, Gunjan Chhablani, Bhavitvya Malik, Simon Brandeis, Teven Le Scao, Victor Sanh, Canwen Xu, Nicolas Patry, Angelina McMillan-Major, Philipp Schmid, Sylvain Gugger, Clément Delangue, Théo Matussière, Lysandre Debut, Stas Bekman, Pierric Cistac, Thibault Goehringer, Victor Mustar, François Lagunas, Alexander M. Rush, and Thomas Wolf. Datasets: A Community Library for Natural Language Processing, 2021.
- [44] Lingling Xu, Haoran Xie, Si-Zhao Joe Qin, Xiaohui Tao, and Fu Lee Wang. Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment, 2023.
- [45] Tom Rainforth, Adam Kosiorek, Tuan Anh Le, Chris Maddison, Maximilian Igl, Frank Wood, and Yee Whye Teh. Tighter variational bounds are not necessarily better. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4277–4285, 2018.
- [46] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635, 2021.

A Detailed Formal Engine Architecture

In this section, we provide the full architectural specification of the seven co-operating components comprising the formal engine introduced in Section 3: the stream buffer, the DFA-based lexer, the stream scanner, the graph-structured stack, the parse lattice, the extended GLR parser, and the engine frontend.

BUFFER. At the upstream end of the engine, the BUFFER exposes a single-character lookahead reader over an append-only stream of characters. The `feed` operation appends a chunk of text, or an end-of-stream (EOS) sentinel, without disturbing the current read offset; this is how the engine ingests the characters of a subword token. The `validate` and `revert` operations together implement a checkpoint mechanism: once a token has been accepted by the PARSER, the BUFFER is validated and its checkpoint advanced to the current write head; if the token is rejected, the BUFFER is reverted

and all characters written past the last checkpoint are discarded. The BUFFER owns the text; both the LEXER and the SCANNER operate on it through a small reader interface exposing offset, lookahead, and the at-end-of-file predicate.

LEXER. The LEXER is a deterministic finite-state recognizer driven by a single compiled transition table. It advances two DFA states in parallel, `plus` and `minus`, which share the same transition function but are seeded from distinct starting states that depend on the parsing context. The `plus` state recognizes the terminals that are valid lookaheads in the current parse state, including any keywords and the generic word terminal whenever the latter is itself admissible. The `minus` state is seeded when the word terminal is *not* a valid lookahead, and while at least one keyword is a valid lookahead, in which case it recognizes the word terminal alone. Together these two states give the engine the information it needs to reproduce Tree-sitter’s keyword extraction. At every step the LEXER reads one character, advances the two DFA states, and records the most recent accepting position as a candidate endpoint, yielding maximal-munch tokenization. The LEXER’s `lex` operation drives this loop from the current reader position: it steps both DFA tracks until both die or the *buffer* is exhausted, returns the last matched symbol together with the accepting position, and writes the surviving DFA states back into the LEXER’s state before returning. When both DFA states accept at a shared endpoint, `plus` acceptance takes precedence over `minus` acceptance. The pair of DFA states constitutes the LEXER’s state, which the PARSER can read and reset to suspend and resume lexing at subword token boundaries when the BUFFER is exhausted mid-lexeme.

SCANNER. Tree-sitter grammars routinely rely on external scanners written in C to recognize tokens that cannot be expressed as regular languages, e.g., indentation in Python, heredocs, raw strings, template literals. The engine integrates such scanners directly through a foreign function interface: it loads the compiled scanner library and exposes the Tree-sitter callback surface. Across scans, the SCANNER’s state is persisted through Tree-sitter’s native serialization protocol: at the end of each scan the state is serialized into a byte string, which the engine carries alongside the LEXER’s state and restores at the start of the next scan. Within a single scan, however, a non-trivial challenge arises because external scanners are written under the assumption of a complete input: when the SCANNER requests the next character past the end of the currently available text, it cannot simply block. We resolve this by running each external scanner invocation inside a *scanlet*, a suspendable coroutine carrying one external scanner call, which yields control back to the PARSER when the external scanner hits the end of the BUFFER. The SCANNER’s `scan` operation either resumes an existing scanlet or spawns a fresh one at the current BUFFER position, seeded from the serialized state and the set of recognizable external symbols, then switches execution into the scanlet; the scanlet runs the C scanner callback directly, yielding back to the PARSER whenever the callback requests a character past the BUFFER boundary, and returns the matched symbol together with the accepting position once the callback completes. Once additional text has been fed, the same scanlet is resumed and continues from exactly the C-side instruction at which it was suspended. Cloning a scanlet duplicates its suspended state, allowing each branching parse hypothesis to carry its own in-flight scan.

STACK. The parser operates over a graph-structured stack (GSS), the standard device for sharing structure across the parallel stacks of a GLR parser. The GSS is a directed acyclic graph, where each path corresponds to one stack. Each node of the GSS carries a parse state and a set of predecessor links. The push operation takes a parse state and embeds it into a new node that points back to the top of one of the parallel stacks. Two nodes carrying the same parse state can be merged by taking the union of their predecessor sets, so parse paths that converge on a common parse state can be represented by a single shared node, while the distinct histories stay separate. The pop operation, parameterized by a count, returns the set of nodes reachable by traversing backwards that many links, so the reduction of a grammar rule can fan out over multiple paths, without ever materializing the underlying parse forest.

LATTICE. A graph-structured stack captures alternative parse stacks, but the engine must also account for alternative lexical futures—suspended LEXER and SCANNER states that may resolve to a lexical token. The engine must additionally track the ordering constraints that maximal-munch tokenization imposes on such futures, and it must keep each lexical hypothesis synchronized with the parse stack it belongs to. This is realized by the LATTICE, which consists of a superposition of lexico-parse hypotheses arranged as a dependency graph. Each lattice vertex records an alternative parse stack, a BUFFER position, a LEXER state, and a SCANNER state. The SCANNER state is either a

serialized Tree-sitter state or a live scanlet when an external scan is in flight. A vertex is keyed by a *signature* consisting of its top parse state, text position, LEXER state, and SCANNER state. Vertices sharing a parent vertex are grouped by their signatures and merged by taking the union of their parse stacks, so lexico-parse branches that reconverge never multiply. The LATTICE is accessed through a set of *heads* that point to its vertices. The *focus* operation designates as *visible*—the active target of lattice operations—the vertex pointed to by a head. The *superpose* operation inserts a new vertex between the visible vertex and its parent; its destructive dual, *collapse*, tears down the subtree beneath the visible vertex. The *create* operation spawns a new vertex under the same parent as the visible vertex; its destructive dual, *annihilate*, removes the visible vertex—reconnecting its children to its parent. The two constructive operations return a fresh head pointing to the new vertex, while the two destructive operations retire the head pointing to the vanished vertex.

PARSER. The PARSER is a GLR parser over the Tree-sitter parse and goto tables, where the collection of parse hypotheses—pairs of a stack version and a buffer position—is extended into the richer DAG structure of the LATTICE. Every parse step opens with a lex step that yields a single potential lookahead by dispatching, in order, to the SCANNER’s *scan* or the LEXER’s *lex* operation. If an external scan is already in flight, the scanlet is resumed; if the LEXER is in a suspended state, it is similarly resumed; otherwise the SCANNER is tried first, and, if it yields nothing, the LEXER is tried—the seed for both is taken from the *lex mode* of the current parse state. When the BUFFER is exhausted mid-lexeme—that is, when the SCANNER returns a scanlet or when the plus DFA track of the LEXER is still alive—the parser inserts a continuation vertex into the LATTICE via the *superpose* operation, carrying the in-progress scanlet or the LEXER’s state so that scanning or lexing can be resumed verbatim once more text arrives. A lexed lookahead (or the LEXER ending in a minus-only state; see below) triggers the *collapse* of the visible parse vertex. The parse vertex is eventually and unconditionally *annihilated*, and if a lookahead is present, the PARSER then drives shifts and reductions. Reductions are serialized via a deduplicating worklist, fanning out across the graph-structured stack at parse-table conflicts and merging derivations that reconverge on a common stack top; a shift creates a new parse vertex. The PARSER cycles through the LATTICE until all hypotheses are either exhausted or paused at the BUFFER boundary. The LATTICE is therefore the locus of the *speculative-lexing* mechanism: within a single parse cycle the parser may commit to a token at one endpoint while a parent hypothesis continues lexing past it, preserving the maximal munch property even when a subword token boundary falls beyond the characters currently in the BUFFER.

When the LEXER returns the word symbol from minus acceptance, the terminal is inadmissible in the current parse state; the parse node is therefore *collapsed* and *annihilated* and no parse action is triggered, integrating Tree-sitter’s keyword extraction—an identifier that merely begins with a keyword is lexed to its full extent and rejected, while a lexeme equal to a contextually valid keyword is shifted as that keyword. When the LEXER ends in a minus-only state at the BUFFER boundary, the PARSER conservatively retires the hypothesis via *collapse* and *annihilate*. This is the only over-approximation the engine makes to the filtering of subword tokens; the filter remains complete for most programming languages, essentially complete for the others, and could be lifted entirely.

FRONTEND. The FRONTEND integrates the six components above into the standard LLM generation loop via rejection sampling. Before each subword token is emitted, the FRONTEND snapshots the current LATTICE. The language model produces next-token logits; the FRONTEND takes the highest-scoring candidate token, decodes it into characters, feeds them to the BUFFER, and runs the PARSER. If at least one hypothesis of the LATTICE survives, the token is accepted: the BUFFER is *validated* and the FRONTEND advances. If every hypothesis dies, the token is rejected: the BUFFER is *reverted*, the LATTICE is restored from its snapshot, the offending token’s logit is set to $-\infty$, and the FRONTEND resamples from the updated distribution.

B Comparison with Alternative Discrete Latent Estimators

To appreciate our choice of the reweighted wake-sleep (RWS) algorithm in Section 4.3, it is instructive to compare it against alternative score-function estimators developed for discrete latent variables.

Unbiased estimators such as VIMCO [32] reduce variance via leave-one-out baselines:

$$\hat{\nabla}_{\phi}^{\text{VIMCO}} = \sum_{k=1}^K [\log \hat{p}(x) - \log \hat{p}_{-k}(x)] \nabla_{\phi} \log q_{\phi}(z_k | x)$$

where $\hat{p}(x) = \frac{1}{K} \sum_j w_j$ and $\hat{p}_{-k}(x)$ is a leave-one-out marginal likelihood estimate. While unbiased, VIMCO suffers from decaying signal-to-noise ratios as the number of particles K increases [45], limiting its utility in multi-sample settings.

More recently, estimators such as ARM [33] and DISARM [34] leverage antithetic sampling over the uniform variables underlying the Gumbel-Max trick to achieve strictly lower variance for categorical latents. In parallel, OVIS [35] derives the theoretically optimal control variate by minimizing the variance of the gradient estimator directly. However, because these estimators scale their gradient updates by quantities that correlate with the magnitude of the unnormalized objective function, they frequently suffer from signal collapse and severe gradient variance near sharp local optima.

In contrast, the wake- ϕ phase of the reweighted wake-sleep algorithm updates the proposal parameters ϕ via the surrogate objective:

$$\mathcal{J}_{\text{wake-}\phi} = - \sum_{k=1}^K \tilde{w}_k \log q_{\phi}(z_k | x), \quad \text{where} \quad \tilde{w}_k = \frac{w_k}{\sum_{j=1}^K w_j}$$

Because the normalized importance weights satisfy $\sum_{k=1}^K \tilde{w}_k = 1$ by construction, the total gradient weight assigned to the proposal update is strictly bounded and invariant to the absolute magnitude of the underlying likelihoods. This effectively acts as a natural trust-region regularization of OVIS, preventing gradient explosion or collapse and ensuring robust convergence even when discrete choices induce major topological shifts in causal attention.

C Next Steps and Research Vision

In this section, we detail practical next steps and long-term research horizons.

C.1 Practical Next Steps

To further minimize constrained decoding overhead, we plan to integrate advanced low-level execution mechanisms, including byte-level lexing, trie-based vocabulary masking, and optimized rejection sampling architectures inspired by high-performance generation engines like Guidance [11] and XGrammar [9]. We also aim to advance lazy compilation by incorporating derivative-based parsing for lexing and fusing Earley parsers [17] with our GLR lattice parser. In parallel, we intend to supplement our engine with the highly efficient incremental parsing capacities offered by Tree-sitter, which parsing based on the GLR algorithm can natively support [25]. Finally, we plan to develop a low-level external stream scanner API to seamlessly accommodate languages requiring arbitrary external state tracking directly over token streams, extending Tree-sitter’s external scanner API to autoregressive generation.

Future empirical work will conduct extensive ablation studies to isolate the mechanisms driving our observed loss reductions. Specifically, we will investigate regimes where full deterministic syntax injection brings no improvement (confirming that teacher-forcing the complete syntax trace provides the same or worse modeling capacity than baseline SFT), as well as evaluating random masking and caching strategies (with and without fixed frequencies) to verify that the gains stem specifically from learned, adaptive latent routing. Crucially, we aim to evaluate our fine-tuned models across standard, widely established benchmarks for code generation to rigorously assess downstream functional correctness and execution accuracy.

A central near-term objective is the bidirectional coupling of our two contributions: deploying the formal engine during generation to actively verify and constrain the model’s latent non-terminal proposals. By validating candidate syntactic non-terminals against the partial GLR parse state before they are emitted, the engine can prune invalid structural derivation paths, ensuring that internal syntactic scratchpads remain provably well-formed.

C.2 Long-Term Research Vision

Toward AI systems that formally plan. Standard autoregressive language models treat highly structured domains—such as source code, formal logic, and mathematical proofs—as flat, one-dimensional sequences of tokens. This absence of explicit hierarchical representations induces well-documented failure modes: syntax hallucinations, inefficient long-horizon planning, and brittle compositional reasoning. Existing neuro-symbolic methods typically intervene only at inference time via rigid rejection sampling, treating formal grammar rules as external constraints rather than internalizing them during training. We view WoFT as a stepping stone toward transforming generative language models from surface-level statistical token predictors into verifiable reasoning systems that natively plan within formal structural abstractions.

Hierarchical and non-linear generation. Autoregressive decoding is traditionally restricted to left-to-right, linear token generation. However, because formal non-terminal symbols encapsulate complete, well-formed syntactic subtrees, they provide a natural foundation for non-linear, hierarchical generation. Rather than predicting every subword token strictly sequentially, an architecture can plan high-level syntactic scaffolding (such as function signatures, loop structures, or control branches) before recursively populating constituent subtrees. Beyond preserving long-range structural coherence across extended contexts, such hierarchical generation opens possibilities for parallelized subtree decoding and modular state reuse across independent syntactic scopes.

Structural working memory and context compression. The discrete hierarchical representations learned by WoFT offer a dual application for memory management in long-horizon reasoning and agentic workflows. As models execute multi-step software engineering or reasoning tasks, context windows quickly become saturated with verbose code and execution traces. Rather than relying on lossy natural-language summaries or ungrounded latent vectors, models can learn to compress past working memory using the exact same formal grammar non-terminals. By abstracting completed functional blocks or verified execution traces into compact abstract syntax tree nodes, the system retains precise structural and semantic fidelity while drastically reducing effective context length.

Generalization to dependent type systems. While our current formal engine and latent formulation operate over compiler-grade Tree-sitter grammars, formal reasoning in pure mathematics and safety-critical software verification requires expressivity beyond context-free formalisms. A natural frontier is extending the WoFT framework to dependent type theories, on which interactive theorem provers such as Lean 4 [46] are built. In formal mathematics, proof construction proceeds through tactic sequences guided by evolving proof states. By formulating proof derivations as latent structural variables, this paradigm can help eliminate deductive hallucinations and help structure proof search.

Learning latent formal structures. In our current framework, formal non-terminals are defined a priori by an externally supplied grammar specification. A compelling theoretical question is whether these formal structures can themselves be inferred and learned. By optimizing the marginal likelihood jointly over grammar production choices and token sequences, future systems could dynamically induce, adapt, and refine latent formal grammars for novel Domain-Specific Languages (DSLs) or under-specified formal domains. This would for example allow an autonomous agent to synthesize its own verifiable, discrete internal languages tailored to specific problem-solving domains.

Continuous and differentiable structural latents. Finally, an intriguing frontier involves relaxing discrete grammar symbols into continuous formal representations while maintaining the symbolic invariants of the formal engine at structural boundaries. By combining the fluid, end-to-end differentiability of continuous latent reasoning with the rigorous boundaries provided by formal grammars, this hybrid approach could realize fully differentiable yet symbolically grounded syntactic planning.